Advanced RAG + LLM Application — Deep Dive

Contents

Advanced RAG + LLM Application — Deep Dive **1**
Table of contents . . . 1
1. One-paragraph summary . . . 2
Part I — Foundations . . . 2
Part II — The big picture . . . 3
Part III — Component deep dives . . . 4
Part IV — Evaluation . . . 8
Part V — Running it . . . 9
Part VI — Security, privacy, and ethics . . . 10
Part VII — Limitations and future work . . . 10
Glossary . . . 11
Appendix A — File-by-file reference . . . 12
Appendix B — The formulas, collected . . . 13

Advanced RAG + LLM Application — Deep Dive

Repository: https://github.com/archit2501/advanced-rag-llm-app

This document explains the project from first principles to implementation detail. It is written to be read top to bottom by someone new to Retrieval-Augmented Generation, but it is also a reference: every subsystem has its own section, and there is a full glossary at the end. Terms are **bolded and defined** the first time they appear.

It describes the code as it actually is *today*, including the pluggable embedder, the honest citation handling (no fabricated markers), the configurable CORS/upload limits, and the learned-embedding ablation. Where the design makes a trade-off or has a known limit, this document says so plainly rather than overselling it.

Table of contents

1. One-paragraph summary
2. Part I — Foundations
3. Part II — The big picture
4. Part III — Component deep dives
5. Part IV — Evaluation
6. Part V — Running it
7. Part VI — Security, privacy, ethics
8. Part VII — Limitations and future work
9. Glossary
10. Appendix A — File-by-file reference
11. Appendix B — The formulas, collected

---

1. One-paragraph summary

This is a **local knowledge assistant**. You give it a set of documents, ask a question in plain language, and it answers using passages it retrieves from those documents — showing you the exact sources it used, and refusing to answer when the documents do not support one. It runs fully offline with no API key by default, using a deterministic keyword-and-vector retrieval pipeline and an extractive answerer; optionally it can call OpenAI or a local Ollama model to generate answers instead. It ships with a 41-question evaluation set, a retrieval-strategy comparison, a FastAPI backend, and a small web UI. The point of the project is not a high accuracy number — it is to make every step of a RAG system *visible and measurable*: ingestion, retrieval, grounding, citations, refusal, and evaluation are separate, inspectable pieces.

Part I — Foundations

1.1 The problem

A **Large Language Model (LLM)** — a neural network trained to predict and generate text — is fluent but has two problems for document question-answering:

1. **It does not know your documents.** Its knowledge is frozen at training time and does not include your organization's internal handbook, this week's policy, or any private corpus.
2. **It can make things up.** An LLM will often produce a confident, plausible-sounding answer even when nothing supports it. A fabricated but convincing statement is called a **hallucination**.

You could *retrain* or *fine-tune* the model on your documents, but that is expensive, must be redone every time a document changes, and still does not let you point at *where* an answer came from.

1.2 The solution: RAG

Retrieval-Augmented Generation (RAG) solves this differently. Instead of putting the knowledge *inside* the model, you keep the model unchanged and, at question time:

1. **Retrieve** the most relevant passages from your document collection.
2. **Augment** the model's prompt by pasting those passages in as context.
3. **Generate** an answer that is instructed to use only that context, and to cite it.

The win: documents can change any time (no retraining), you can *show the sources* behind every answer, and you can measure retrieval quality separately from answer quality.

> **RAG vs. fine-tuning.** Fine-tuning changes the model's weights from training examples; RAG changes the model's *input* at inference time. For a document assistant, RAG is usually the right tool because the knowledge is dynamic and provenance matters. They are complementary, not competitors.

1.3 The vocabulary you need up front

Term	Plain definition
Chunk	A bounded passage (here, up to ~900 characters) cut from a larger document so it can be individually retrieved and cited.
Embedding	A list of numbers (a *vector*) representing a piece of text, so that similar texts have nearby vectors.
Retrieval	Finding and ranking the chunks most relevant to a question.
Grounding	Making the answer rest on retrieved evidence rather than the model's memory.
Citation	A pointer (`[1]`, `[2]`, …) from a sentence in the answer back to the source chunk it came from.

Term	Plain definition
Refusal	Deliberately declining to answer when the retrieved evidence is too weak, instead of guessing.
Hallucination	An answer that is unsupported by or contradicts the available evidence.

Everything below is an expansion of how this project does each of those things.

Part II — The big picture

2.1 Module map

The system is a pipeline of small, single-responsibility modules. Each can be tested or swapped without rewriting the others.

Layer	Responsibility	File
Configuration	Read env vars and defaults into a frozen `Settings` object	`backend/app/settings.py`
Ingestion	Extract text, normalize, assign stable IDs, split into chunks	`backend/app/text_processing.py`
Embeddings	Turn text into vectors (hashing default, optional learned model)	`backend/app/embeddings.py`
Storage	Persist documents, chunks, vectors, metadata in SQLite	`backend/app/store.py`
Retrieval	Rank chunks by keyword (BM25) + vector (cosine), fused with RRF	`backend/app/retrieval.py`
Generation	Produce the answer (offline extractive / OpenAI / Ollama)	`backend/app/llm.py`
Orchestration	Wire ingestion → retrieval → refusal → generation → citations	`backend/app/rag.py`
API	HTTP endpoints + serve the frontend	`backend/app/api.py`
Frontend	Upload, chat, source panel, evaluation controls	`frontend/`
Evaluation	Score answers + compare retrieval strategies	`backend/app/evaluation.py`, `backend/app/comparison.py`

2.2 What happens when you add a document (ingestion flow)

```
file bytes
  → extract_text()        decode text / extract PDF, then normalize whitespace
  → stable_id()           SHA-256(filename + text)[:16]  → deterministic document id
  → chunk_text()          pack paragraphs to ~900 chars with 160-char overlap
  → embedder.embed_many() one vector per chunk
  → store.upsert_document()  write document + chunks + vectors + metadata to SQLite
```

Every chunk keeps its `document_id`, `source` filename, `title`, and zero-based `position`. That chain of provenance is what later makes citations possible.

2.3 What happens when you ask a question (query flow)

This is the heart of the system, and it reflects the **current** code in `rag.py:ask()`:

```
question
  → store.all_chunks()               load every chunk from SQLite
  → retriever.search(q, chunks, k)   score all chunks, return top-k
  → _should_refuse(results)?         evidence gate
        yes → return canned "not available" answer + diagnostics(refused=True)
        no  ↓
  → llm.generate(q, results)         provider produces raw answer text
  → strip_invalid_citations()        delete any [n] outside the valid range
  → cited_numbers()                  list which valid sources were actually cited
  → uncited = results exist AND nothing was cited   (recorded as a diagnostic)
  → build citation objects for the cited sources only
  → return { answer, citations, sources, diagnostics }
```

Two things worth noting, because they were deliberate design corrections:

- **No fabricated citations.** Earlier the code appended a fake `[1]` whenever the model forgot to cite, which manufactured a perfect "citation coverage" number. That is gone. Citations now reflect only what the model *actually* cited; if it cited nothing, the response carries an `uncited: true` diagnostic instead of a fake marker.
- **The offline answerer always cites** (it stitches together sentences and tags each with its source number), so in offline mode `uncited` is effectively never set — but the honest machinery is there for real LLM providers.

Part III — Component deep dives

Each subsystem below follows the same shape: *what it is → how this project does it → the trade-off.*

3.1 Configuration — `settings.py`

What. A single frozen `@dataclass` called `Settings` holds every tunable value, populated from environment variables (via a `.env` file) with sensible defaults. Frozen means it cannot be mutated after construction, which keeps behavior predictable across a request.

Key fields (current):

Field	Env var	Default	Meaning
`chunk_size`	`CHUNK_SIZE`	`900`	Target max characters per chunk
`chunk_overlap`	`CHUNK_OVERLAP`	`160`	Characters repeated across chunk boundaries
`embedding_dim`	`EMBEDDING_DIM`	`384`	Hashing-vector dimension
`top_k`	`TOP_K`	`5`	Chunks kept per query
`min_hybrid_score`	`MIN_HYBRID_SCORE`	`0.10`	Evidence-gate threshold
`max_refusal_keyword_score`	`MAX_REFUSAL_KEYWORD_SCORE`	`2.0`	Evidence-gate keyword condition
`embedding_provider`	`EMBEDDING_PROVIDER`	`hashing`	`hashing` or `sentence-transformers`
`embedding_model`	`EMBEDDING_MODEL`	`sentence-transformers/all-MiniLM-L6-v2`	Learned model (when provider != hashing)
`cors_allow_origins`	`CORS_ALLOW_ORIGINS`	`http://127.0.0.1:8000,http://localhost:8000`	Browser origins allowed to call the API
`max_upload_mb`	`MAX_UPLOAD_MB`	`10`	Upload size ceiling
`llm_provider`	`LLM_PROVIDER`	`offline`	`offline`, `openai`, or `ollama`

Field	Env var	Default	Meaning
`openai_model`	`OPENAI_MODEL`	`gpt-4o-mini`	OpenAI model id
`ollama_model`	`OLLAMA_MODEL`	`llama3.1:8b`	Ollama model tag

Trade-off. Central config makes the system easy to tune, but several of these numbers (the gate thresholds especially) are *calibrated to this corpus* and are not universal — see §3.6.

3.2 Ingestion and chunking — `text_processing.py`

Text extraction. Plain-text formats (`.txt`, `.md`, `.csv`, `.json`, `.html`, `.py`) are decoded as UTF-8 with replacement for bad bytes. PDFs are supported when the optional `pypdf` package is installed. Extracted text is then **normalized**: null bytes removed, line endings unified, runs of spaces collapsed, and 3+ blank lines reduced to a paragraph break.

Stable IDs. A document's id is `SHA-256(filename + "::" + text)[:16]`. Because it is derived from content, re-ingesting the same file is idempotent (it replaces, not duplicates). Each chunk id is `document-id#chunk-0000`.

Chunking algorithm. *Chunking* is splitting a document into retrievable passages. Chunk size is a trade-off: too small loses the context needed to interpret a fact; too large mixes topics and dilutes ranking. This project:

1. Splits normalized text into paragraphs.
2. Packs adjacent paragraphs together until the next one would exceed `chunk_size` (900).
3. When it starts a new chunk, it carries a **tail overlap** (up to `chunk_overlap` = 160 chars) from the previous chunk, so a fact spanning a boundary is not lost.
4. Any single paragraph larger than `chunk_size` is split on word boundaries, also with overlap.

Trade-off. The chunker is deterministic and easy to reason about, but it is structure-blind: it does not understand headings, tables, or page layout, and there is no OCR for scanned PDFs.

3.3 Embeddings — `embeddings.py`

What an embedding is. A vector that represents text so that related texts land near each other. Good embeddings let a query about "pay for interns" match a passage that says "stipend," even with no shared words.

The default: `HashingEmbedder` (384-dim, deterministic, offline). For each token it computes a BLAKE2b hash, uses that to pick a vector position and a sign, and accumulates a weight of `1 + log(count)`; the vector is then L2-normalized. This needs no model download and gives identical results every run.

> **Honest caveat.** This is *not* a learned semantic model. It is essentially a signed bag-of-words projection: it preserves token identity but cannot capture paraphrase or synonymy well. Its "vector similarity" is therefore strongly lexical. It exists so the whole pipeline runs with zero dependencies — not to compete with a real embedding model.

The optional upgrade: `SentenceTransformerEmbedder`. Setting `EMBEDDING_PROVIDER=sentence-transformers` swaps in a learned model (`all-MiniLM-L6-v2`, also 384-dim) behind the *same interface*. The import is guarded, so the default install stays lightweight. A factory function `make_embedder(settings)` picks the implementation; `rag.py` calls that factory and never hard-codes a class. This is what makes the embedder genuinely pluggable.

Cosine similarity. Relevance between two vectors is measured with **cosine similarity** — the cosine of the angle between them, `Σ a_ib_i / (||a||||b||)`. Because vectors here are pre-normalized, this reduces to a dot product. Negative scores are clamped to zero for the retrieval signal.

3.4 Storage — `store.py`

What. `SQLiteRAGStore` persists two tables in a local file (`.rag_data/rag_index.sqlite3`): `documents` and `chunks`. Each chunk row stores its text, its embedding **as a JSON string**, and its metadata. **SQLite** is a serverless relational database — a single file, no server process — which is ideal for a single-user demo.

Trade-off (important). There is **no vector index**. On every query, `all_chunks()` reads *all* chunk rows and JSON-parses *all* embeddings, and the retriever then scores them in Python. This is a brute-force linear scan — perfectly fine for the demo's ~29 chunks, but it does not scale to large corpora or high concurrency. A production system would use a real **vector database** or an **approximate-nearest-neighbour (ANN)** index. The documentation calls this a "linear-scan vector store," not a "vector index," precisely so the term is not oversold.

3.5 Retrieval — `retrieval.py`

Retrieval combines two independent relevance signals and fuses their *rankings*.

Tokenization. Text is lowercased, split into alphanumeric tokens, and a small stopword list is dropped. **Lexical overlap** for a chunk is the fraction of distinct query terms it contains.

Keyword signal — BM25. BM25 is a classic ranking function that rewards chunks containing the query terms, weights rarer terms more via **inverse document frequency (IDF)**, and normalizes for chunk length. This project uses the standard `k1 = 1.5`, `b = 0.75`, with `idf = log(1 + (N - df + 0.5)/(df + 0.5))`.

Vector signal — cosine. The clamped cosine similarity between the query embedding and each chunk embedding (§3.3).

Fusion — Reciprocal Rank Fusion (RRF). Rather than adding raw scores that live on different scales, **RRF** combines the two *ranks*:

```
rrf(chunk) = 1/(60 + vector_rank) + 1/(60 + keyword_rank)
final_score = rrf(chunk) + 0.15 × lexical_overlap
```

A chunk ranked highly by *both* signals gets the strongest fused score. Chunks are sorted by `final_score` and the top-k returned. Each result keeps its fused score, vector score, keyword score, lexical overlap, and rank — so retrieval failures are diagnosable rather than opaque.

Trade-off. RRF with `k = 60` and a `0.15` overlap weight is a reasonable, explainable baseline — but those constants should be tuned on a validation split, not chosen by intuition. And with the *hashing* embedder, the vector and keyword signals are correlated (both lexical), which limits how much "hybrid" actually adds. That correlation is exactly what the learned-embedding ablation in §4.5 examines.

3.6 Evidence gate and refusal — `rag.py`

Why. A ranker always returns *something*, even when every candidate is weakly related. Without a check, the generator would treat a bad-but-best chunk as evidence and produce an unsupported answer. An **evidence gate** inspects retrieval quality *before* generation and refuses when it is too weak.

The rule (`_should_refuse`). Refuse if there are no results, or if the best result is weak on *all four* signals at once:

```
best.score < min_hybrid_score (0.10)
  AND best.vector_score < 0.20
  AND best.lexical_overlap <= 0.50
  AND best.keyword_score <= max_refusal_keyword_score (6.0)
```

Requiring all four reduces the chance that one odd score wrongly rejects a good answer. On refusal the system returns *"The answer is not available from the indexed documents."* plus the retrieved diagnostics, so you can still see what was considered.

Trade-off (and a key finding). These thresholds were **calibrated on this corpus** and are tied to the *hashing* embedder's score scale. They do not transfer to a different embedder — the ablation in §4.5 shows the gate collapsing to 0% correct abstention under learned embeddings, because that model's cosine values occupy a different range. The honest takeaway: **refusal thresholds are embedder-specific and must be re-tuned per model on held-out data.**

3.7 Generation providers — `llm.py`

All three providers share one interface: given a question, the retrieved results, and settings, return answer text. That **provider abstraction** is why orchestration code never changes when you switch providers.

Offline extractive (default, `offline-extractive`). No LLM, no network. For each retrieved chunk it selects the sentence with the greatest overlap with the query terms, takes up to three distinct such sentences, and tags each with its source number (`… [2]`). Deterministic, free, private, test-friendly — but it copies and stitches rather than truly synthesizing or paraphrasing.

OpenAI Responses API (`LLM_PROVIDER=openai`). Uses the OpenAI Python SDK's Responses API: system instructions passed separately from the user context, a configurable model (default `gpt-4o-mini`), and a max-output-token budget. If the model id starts with `gpt-5`, it additionally sends a reasoning-effort setting. The API key comes from `OPENAI_API_KEY` and is never committed. (No live OpenAI call was made in the recorded results; the adapter is covered by a mocked unit test.)

Ollama (`LLM_PROVIDER=ollama`). Sends a non-streaming request to a local Ollama server (`http://localhost:11434`, model `llama3.1:8b`) at low temperature. Keeps generation on-device.

The grounded prompt. For accepted retrieval, each result becomes a numbered context block (`[1] title (source, chunk N)` + text). The system instructions tell the model to answer *only* from context, cite as `[1] [2] …`, say the answer is unavailable if context is insufficient, and stay concise. This improves traceability; it does not make hallucination impossible.

3.8 Citations — `llm.py` helpers + `rag.py`

Two small pure functions enforce citation hygiene:

- `strip_invalid_citations(answer, n)` deletes any `[k]` where `k` is outside `1..n` (the number of retrieved results).
- `cited_numbers(answer, n)` returns the distinct valid citation numbers actually present.

`rag.py` then builds citation objects only for those numbers, and sets the `uncited` diagnostic when results existed but nothing was cited. **Citation presence is not citation correctness** — a `[2]` marker proves formatting, not that source 2 supports the claim. Claim-level entailment checking is listed as future work.

3.9 API — `api.py`

FastAPI is a typed Python web framework. Endpoints:

Method + path	Purpose
`GET /`	Serve the frontend `index.html`
`GET /api/health`	Status, document/chunk counts, provider
`GET /api/documents`	List indexed documents
`DELETE /api/documents`	Clear the index
`POST /api/documents`	Upload a file (now rejected with **413** above `max_upload_mb`)
`POST /api/ingest-path`	Ingest a server-side path or the sample corpus
`POST /api/ask`	Ask a question → answer + citations + sources + diagnostics
`POST /api/evaluate`	Run the bundled evaluation set

CORS is now built from `settings.cors_allow_origins` (an explicit localhost allow-list) rather than a wildcard, so credentialed requests are valid and the surface is narrower. Errors (empty question, unsupported type, oversize upload) become clean HTTP status codes.

3.10 Frontend — `frontend/`

Plain HTML/CSS/JS (no framework). It provides upload, sample-load, clear, a chat box, a source panel with score and excerpt per citation, a live health/latency badge, and an "evaluate" button. All rendered values are HTML-escaped before insertion, so document text cannot inject markup (basic XSS safety). It is served from the same FastAPI process to keep setup to one command.

Part IV — Evaluation

4.1 Why RAG evaluation must be split

RAG quality is not one number. Retrieval can find the right evidence while generation writes a poor answer; or generation can sound fine while retrieval missed the key fact. So the project measures **retrieval quality**, **answer quality**, **citation/refusal behavior**, and **latency** separately.

4.2 The dataset

A versioned **JSON Lines (JSONL)** file — one JSON object per line, easy to diff and extend — with **41 cases** over 13 synthetic internship documents:

- 30 **answerable**, 3 **ambiguous**, 3 **conflicting-policy**, 5 **unanswerable**.

Each case carries an id, question, expected answer, expected source documents, required-fact phrases (where applicable), scenario, and difficulty. The corpus deliberately includes an archived 2025 FAQ alongside current policies so the system faces source conflict and recency.

4.3 The metrics, defined

- **Required-fact recall** — fraction of a case's expected fact phrases that appear in the answer. Uses a lenient lexical proxy (a phrase counts if >= half of its words longer than 3 characters appear). Easy to run; blind to meaning, negation, and claim boundaries.
- **Citation-or-refusal coverage** — 1 if the response has >= 1 citation *or* it refused. Structural, not semantic.
- **Unanswerable-refusal accuracy** — of the truly unanswerable cases, the fraction correctly refused.
- **Answerable non-refusal rate** — of the answerable cases, the fraction *not* wrongly refused.
- **Recall@k** — is an expected source among the top-k retrieved sources?
- **MRR (Mean Reciprocal Rank)** — rewards the first relevant result appearing early.
- **nDCG@k (Normalized Discounted Cumulative Gain)** — rewards relevant results near the top, normalized against the ideal ordering.
- **Benchmark score** — a project-defined blend of ranking and abstention behavior, used to pick a baseline on *this* corpus only.

4.4 Recorded results — hashing baseline (offline)

Answer evaluation:

Metric	Value
Cases / fact-scored	41 / 36
Average required-fact recall	**0.8241**
Citation-or-refusal coverage	1.0000
Answerable non-refusal rate	1.0000
Unanswerable-refusal accuracy	0.8000
Mean latency	~4.6 ms

Retrieval comparison:

Configuration	Recall@5	MRR	nDCG@5	Abstention	Combined
Vector-only	0.9583	0.9028	0.9033	0.6000	0.8663
Keyword-only	1.0000	0.9306	**0.9441**	0.0000	0.8290
Hybrid	1.0000	0.9097	0.9283	0.8000	**0.9126**

Read this honestly: keyword-only actually has the *highest* nDCG@5. Hybrid "wins" the combined score only because the score folds in abstention accuracy, where the calibrated gate refused four of five unsupported questions. That is an engineering trade-off, not proof of semantic superiority — and because the hashing vector signal is itself lexical, "hybrid" here is largely "lexical + a refusal policy."

4.5 Recorded results — learned-embedding ablation

Re-running with `all-MiniLM-L6-v2` (`EMBEDDING_PROVIDER=sentence-transformers`), same corpus and cases:

Configuration	Recall@5	MRR	nDCG@5	Abstention	Combined
Vector-only	0.9861	0.9398	0.9365	0.0000	0.8223
Keyword-only	1.0000	0.9306	0.9441	0.0000	0.8290
Hybrid	1.0000	0.9236	0.9408	0.0000	0.8260

Answer eval under the swap: fact recall **0.8056**, unanswerable-refusal **0.0000**, latency ~**29 ms**. Three findings, each more instructive than a headline number:

1. **Learned dense retrieval really does rank better.** Vector-only nDCG@5 rises 0.9033 → 0.9365 and Recall@5 → 0.9861 — confirming that under hashing the vector and keyword channels were near-duplicate lexical signals, and a learned model makes the dense channel a distinct, stronger ranker.
2. **The gain doesn't reach the answer metric on this corpus.** Extractive fact recall actually slips (0.8241 → 0.8056) and keyword-only still leads on nDCG. This hand-written corpus shares vocabulary between questions and evidence, so lexical matching is near-ceiling; dense retrieval pays off most under paraphrase, which this corpus underrepresents.
3. **The evidence gate does not transfer.** Abstention collapses to 0.0 for every mode, because the thresholds were calibrated to the hashing score scale. Refusal thresholds are embedder-specific and must be recalibrated per model.

Practical conclusion: keep hashing as the zero-dependency default; treat learned embeddings as an opt-in that requires threshold recalibration (plus ~6× query latency and a heavy dependency) before its retrieval gains translate end to end. Raw outputs live in `evals/comparison_results_sbert.json` and `evals/results_sbert.json`.

4.6 Validity boundaries

These are development measurements on a small synthetic corpus, not production claims. Fact recall is a term-overlap proxy; the gate was tuned on the same data it is measured on; citation coverage measures presence, not support; latency excludes network, ingestion, and remote-provider time; and no paid LLM was in the measured run. A credible research claim would need held-out validation/test splits and human-reviewed groundedness.

Part V — Running it

5.1 Prerequisites

- Python 3.9+, a virtual environment.
- Dependencies from `requirements-dev.txt`.
- Optional: `pypdf` (PDF ingestion), `sentence-transformers` (learned embeddings), an OpenAI key or a running Ollama server.

5.2 Install and run

```
git clone https://github.com/archit2501/advanced-rag-llm-app.git
cd advanced-rag-llm-app
python -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
cp .env.example .env
make dev            # then open http://127.0.0.1:8000
```

5.3 Reproducible commands

```
make test           # unit tests (9)
make eval           # answer evaluation → evals/results.json
make compare        # retrieval comparison → evals/comparison_results.json
make test-ui        # Playwright browser smoke test (server must be running)

# learned-embedding runs:
pip install sentence-transformers
python -m backend.app.comparison --embedding-provider sentence-transformers \
  --output evals/comparison_results_sbert.json
EMBEDDING_PROVIDER=sentence-transformers python -m backend.app.evaluation \
  --reset --output evals/results_sbert.json
```

Part VI — Security, privacy, and ethics

Security posture. Built for trusted local, single-user use. There is no authentication, authorization, or tenant isolation; the `DELETE /api/documents` and reset endpoints are unauthenticated by design and must not be exposed publicly. CORS is now an explicit localhost allow-list and uploads are size-capped, but the path-ingestion endpoint should be restricted to an allow-listed directory before any exposure.

Sensitive data + external providers. Before sending any document or question to OpenAI, decide whether that data may leave the environment (classification, retention, region, consent). API keys belong in env vars or a secret manager — never in the repo, prompts, screenshots, or documents.

Prompt injection. Prompt injection is when untrusted content (e.g., text inside an ingested document saying "ignore your instructions") tries to hijack the model. RAG can carry such text straight into the generation context. This project separates instructions from context but does not yet detect malicious content; future controls include content filtering, provenance labels, and prompt-injection test cases.

Ethical use. Sources and refusals improve transparency but do not remove bias, staleness, or over-trust. Treat the assistant as decision support, not authority.

Part VII — Limitations and future work

Current limitations. Small synthetic corpus; hashing embedder is lexical; per-query linear scan doesn't scale; citations are range-validated but not entailment-checked; fact-recall is a lexical proxy; gate thresholds don't transfer across embedders; no reranker, query rewriting, conversation memory, OCR, or table parsing; production security/observability intentionally absent; remote-provider cost/latency/failures unmeasured.

Recommended next steps. - **Held-out data:** validation/test splits with human-reviewed relevance and answer labels. - **Recalibrated learned-embedding run:** re-tune the gate to the learned model's score scale, then re-measure abstention (the 0.0 in §4.5 is uncalibrated gating, not a retrieval limit). - **Reranker:** add a cross-encoder second stage. - **Claim-level grounding:** entailment checks tying each citation to the sentence it supports. - **Provider measurement:** real OpenAI/Ollama runs with recorded model, tokens, cost, latency

percentiles, errors. - **Scale + security:** ANN index, incremental ingestion, auth/RBAC, audit logs, rate limiting.

Glossary

ANN (Approximate Nearest Neighbour) — Fast similarity search over vectors that trades a little exactness for speed; not used here (the corpus is small).

API (Application Programming Interface) — A defined way for one program to request services from another.

BM25 — A keyword ranking function combining term frequency, IDF, and length normalization.

Chunk — A bounded passage cut from a document for indexing, retrieval, and citation.

Chunk overlap — Text repeated between adjacent chunks so boundary-spanning meaning is not lost.

Citation — A pointer from an answer statement to the source chunk supporting it.

Citation accuracy — The fraction of citations that actually support the claim they mark (stronger than mere presence).

Context window — The maximum input+output an LLM can process at once; RAG manages it by sending only top-k chunks.

CORS (Cross-Origin Resource Sharing) — Browser rules for which web origins may call an API; configured here to a localhost allow-list.

Cosine similarity — Similarity based on the angle between two vectors; a dot product for normalized vectors.

Embedding — A numeric vector representing text for similarity search.

Embedding dimension — The length of an embedding vector (384 here).

Evidence gate — A pre-generation check that refuses when retrieval signals are too weak.

Extractive answer — An answer built by selecting sentences from sources rather than generating new prose.

FastAPI — A typed Python web framework used for the backend.

Fine-tuning — Adjusting a model's weights on new examples; contrasted with RAG, which changes the input, not the weights.

Grounding / groundedness — The degree to which answer claims are supported by retrieved sources.

Hallucination — A generated statement unsupported by or contradicting the evidence.

Hybrid retrieval — Combining vector/semantic and keyword/lexical signals.

IDF (Inverse Document Frequency) — A weight that raises the importance of rarer terms.

Inference — Running a trained model to get output (as opposed to training it).

JSON Lines (JSONL) — A file with one JSON object per line; used for the evaluation set.

Keyword retrieval — Search by literal term matches (e.g., BM25).

Latency — Time taken by an operation.

LLM (Large Language Model) — A neural network trained to predict and generate text tokens.

make_embedder — The factory function that returns the hashing or learned embedder based on `embedding_provider`.

Metadata — Data describing another item (chunk title, source, position, character count, …).

MRR (Mean Reciprocal Rank) — Average of 1/rank of the first relevant result.

nDCG@k (Normalized Discounted Cumulative Gain) — A ranking metric rewarding relevant items near the top, normalized to the ideal ordering.

Normalization (vector) — Scaling a vector to unit length so cosine reduces to a dot product.

Normalization (text) — Cleaning whitespace, line endings, and control characters before chunking.

OCR (Optical Character Recognition) — Converting text in images/scans into machine-readable characters; not implemented.

Ollama — A local model-serving runtime; optional generation provider.

Prompt — The instructions plus context sent to an LLM.

Prompt injection — An attack where untrusted text tries to override system instructions.

Provenance — The traceable origin (document, position) of a chunk; the basis for citations.

Provider abstraction — A shared interface letting the app switch generation backends without rewiring orchestration.

RAG (Retrieval-Augmented Generation) — Retrieving external content and using it as generation context.

Recall@k — Fraction of queries where a relevant result appears within the top-k.

Reciprocal Rank Fusion (RRF) — Combining multiple rankings via `1/(k + rank)` contributions.

Refusal — Declining to answer when evidence is insufficient.

Reranker — A second-stage model that reorders initial candidates more accurately.

Retrieval — Finding and ranking source passages relevant to a query.

SQLite — A serverless, single-file relational database.

Stable ID — A content-derived identifier (here SHA-256 prefix) making ingestion idempotent.

Stopword — A very common word dropped during keyword processing.

Token — A unit of text used by a tokenizer or model.

Tokenization — Splitting text into tokens.

Top-k — The number of top-ranked chunks kept (default 5).

Traceability — Connecting an output back to the data, steps, and sources that produced it.

uncited (diagnostic) — A flag set when an accepted answer exists but the model cited nothing; replaces the old fabricated `[1]`.

Uvicorn — The ASGI server that runs FastAPI in development.

Vector database — Storage optimized for embeddings and similarity search; here replaced by SQLite + linear scan.

Vector search — Retrieval comparing a query embedding to chunk embeddings via cosine similarity.

Appendix A — File-by-file reference

File	What lives here
`backend/app/settings.py`	`Settings` dataclass + env parsing
`backend/app/text_processing.py`	`extract_text`, `normalize_text`, `chunk_text`, `stable_id`, file discovery

File	What lives here
backend/app/embeddings.py	tokenize, HashingEmbedder, SentenceTransformerEmbedder, make_embedder, cosine, normalize
backend/app/store.py	SQLiteRAGStore — schema, upsert, list, all_chunks, count
backend/app/retrieval.py	HybridRetriever — BM25, cosine, RRF fusion
backend/app/llm.py	Prompt builders, OfflineLLM, OpenAIResponsesLLM, OllamaLLM, make_llm, citation helpers
backend/app/rag.py	RAGService — ingest orchestration, ask, evidence gate, diagnostics
backend/app/api.py	FastAPI app, endpoints, CORS, upload cap
backend/app/models.py	DocumentChunk, RetrievalResult dataclasses
backend/app/evaluation.py	run_evaluation, JSONL loading, answer metrics
backend/app/comparison.py	run_comparison — vector/keyword/hybrid ablation, Recall/MRR/nDCG
frontend/	index.html, app.js, style.css
evals/	sample_questions.jsonl, recorded *.json results (hashing + sbert)
docs/	Architecture, evaluation, final report, this deep dive
tests/	Unit tests for chunking, ingestion→answer, evaluation, OpenAI provider

Appendix B — The formulas, collected

Hashing embedding (per token): position = `hash % dim`, sign = ±1 from a hash bit, weight = `1 + log(count)`; vector L2-normalized.

Cosine similarity: `cos(a,b) = Σ a_ib_i / (||a||||b||)` → dot product when normalized.

BM25 (per query term in a chunk):

```
idf = log(1 + (N − df + 0.5)/(df + 0.5))
term_score = idf × (f × (k1+1)) / (f + k1 × (1 − b + b × (len/avglen)))
with k1 = 1.5, b = 0.75
```

Hybrid fusion:

```
rrf = 1/(60 + vector_rank) + 1/(60 + keyword_rank)
final = rrf + 0.15 × lexical_overlap
```

Evidence gate (refuse when all hold):

```
score < 0.10  AND  vector < 0.20  AND  overlap <= 0.50  AND  keyword <= 6.0
```

nDCG@k: `DCG = Σ rel_i / log_2(i+1)`, normalized by the ideal DCG.

MRR: average of `1 / rank_of_first_relevant`.